Prompt Engineering 横纵分析报告

> 从"敲魔法咒语"到"设计 AI 交互系统"，提示工程如何长成 AI 时代的核心技能。

一、一句话定义

Prompt Engineering（提示工程）是**设计、优化和管理大语言模型输入指令的系统化方法**。它的核心矛盾在于：模型的输出质量高度依赖输入格式和内容，但模型对 prompt 措辞的敏感性背后，至今没有一套完整的理论解释——实践上极其有效，理论上仍在追赶实践。

> 🎯 **读完这篇你能**：写出有效的 few-shot prompt，理解 Few-shot、CoT、ReAct 各自适用什么场景，能判断什么时候该让模型"一步步思考"、什么时候直接给答案更高效。

阅读指南

如果你只有 5 分钟：读一句话定义 + 横纵交汇洞察的 5.7（"第一性原理追问"）。你能在 5 分钟内理解 Prompt Engineering 到底是在"说话"还是"编程"，以及它的未来方向。

如果你想理解 Prompt Engineering 但不碰技术细节：读技术背景的核心矛盾 + 纵向分析每个阶段的开篇段 + 横向分析的工具分类矩阵。约 20 分钟。

如果你是 Prompt Engineer / 研究者：直接跳纵向分析的 3.3（CoT/ReAct/ToT 推理策略大爆发）和 3.5（DSPy 与自动化工程），以及横向分析的 DSPy vs 手工对比。3.4（Prompt Injection 与安全攻防）是所有部署 Prompt 系统的人必须读的部分。

二、技术背景

核心矛盾：模型的"输出"从根本上不可控

大语言模型在训练中学到的是 token 序列的统计分布——给定上文，预测下一个最可能的词。由此带来三个关键推论：

1. **格式敏感**：同样的问题，措辞换几个字、示例顺序变一下、用 Markdown 还是 JSON——输出质量可能差 76 个百分点（FormatSpread, ICLR 2024）。
2. **没有"理解"只有"匹配"**：模型不是在"理解"你的指令，而是在训练数据中寻找最接近的模式来"续写"。

3. **涌现的表现依赖设计的输入**：CoT、ReAct 等推理策略说到底不是"教模型新能力"，而是"通过特定的输入格式释放模型已经具备的能力"。

Prompt Engineering 的所有工作围绕一个核心洞察：**模型的输出上限由 prompt 定义，但 prompt 和输出之间的映射关系是非线性的、非直观的。**

> 前置知识请参考独立文档 《Prompt_Engineering_前置知识》，涵盖 Zero-shot、Few-shot、CoT、ICL、System Prompt、Prompt Injection 等基础概念的通俗讲解。

> 📚 **关联报告**

> - AI Agent — ReAct/CoT 等提示策略是 Agent 推理循环的起源

> - Context Engineering — Prompt Engineering 正从"写提示词"演化为系统的上下文设计工程

> - LLM 基础大模型 — 模型能力是提示工程的上限，更好的模型需要更好的提示来释放能力

三、纵向分析：从手工模板到自动化工程

3.1 前 GPT-3 时代：手工模板与语言模型适配（2018-2020）

在 GPT-3 定义 "in-context learning" 范式之前，prompt 的概念已经在 NLP 中存在了——但形态完全不同。

BERT 时代的 cloze 模板（2018-2020）：BERT 是掩码语言模型，它的训练任务是预测 `[MASK]` 位置的词。要用 BERT 做分类，就需要把任务包装成 cloze 形式：

```
输入：[CLS] 这部电影非常 [MASK]。 [SEP]
输出：好（表示正面情感）
```

这就是最早的 prompt——**手工设计输入模板来适配预训练模型的输出格式。**

AutoPrompt（2020.11, arXiv:2010.15922） 是这个阶段最有前瞻性的工作。Shin 等人用梯度方法自动搜索 BERT 的最佳 cloze prompt 模板——在情感分析、NLI 等任务上显著超越手工设计。它已经在做"自动搜索最优 prompt"这件事了。

这个阶段的本质是：**prompt 的存在是因为模型架构的局限**（MLM 需要特定输入格式），而不是为了让模型"理解"任务。

3.2 GPT-3 与 In-Context Learning：催化剂的诞生（2020）

2020 年 6 月，GPT-3（arXiv:2005.14165） 改变了所有规则。Brown 等人的核心发现是：

- **规模带来涌现**：模型越大，在 prompt 中给几个示例（few-shot）后完成新任务的能力越强
- **不需要梯度更新**：仅仅在输入中放几个"输入→输出"对，模型就能"学会"执行任务
- **这对当时的 NLP 社区是一个震撼**——长期以来"监督学习需要标注数据"是基本信条，GPT-3 说：不需要，写几句 prompt 就行

GPT-3 的 few-shot 能力直接催生了 Prompt Engineering 这个领域——不是因为 OpenAI 发布了 prompt 教程，而是**它第一次让"怎么写输入"变得跟"怎么训练模型"一样重要**。

同期，**指令微调**的线也在发展——FLAN（2021.09）和 InstructGPT（2022.01）通过在指令数据集上微调模型，让模型更擅长理解和遵循自然语言指令。这条线和 prompt engineering 是互补的：指令微调让模型"更容易听懂"，prompt engineering 研究"该怎么对它说"。

2021 年的 Soft Prompt（连续 prompt） 走了另一条路：Prefix-Tuning（Li & Liang, ACL 2021）和 Prompt Tuning（Lester et al., arXiv:2104.08691）不在输入层加示例，而是加**可学习的连续向量**。这说到底是用梯度下降优化 prompt，而不是用自然语言写 prompt。它在学术上有影响，但与后来主流的"自然语言 prompt"不同路——它更接近微调的一种变体。

3.3 推理策略大爆发：CoT、ReAct、ToT 接连登场（2022）

2022 年是 Prompt Engineering 从"写示例"到"设计推理策略"的转折年。四篇论文在这一年密集出现，各自定义了一个子方向。

Chain-of-Thought（2022.01, arXiv:2201.11903）：Jason Wei 和 Google 团队的发现简单到让人怀疑——在大模型（~100B+）的 prompt 中放几个带推理步骤的示例，模型就会在回答时也生成推理步骤。在 GSM8K（小学数学）上，PaLM 540B 的准确率从标准 few-shot 的 17.9% 飙升到 58.1%。

CoT 的贡献不在于"找到了一种更好的写法"，而是**证明了模型具备潜在的推理路径生成能力，只是需要一个格式来触发它**。这个发现的重要性怎么强调都不过分——它意味着模型内部包含了远超过它在 zero-shot 下表现出的能力。

Zero-shot CoT（2022.05, arXiv:2205.11916） 进一步把门槛降到零——只需在 prompt 末尾加一句 "Let's think step by step"，模型就会做多步推理。在 MultiArith 上，从 17.7% 直接到 78.7%。

"Let's think step by step" 可能是 AI 史上最有影响力的七个英文词。

Self-Consistency（2022.03, arXiv:2203.11171） 不改变 prompt 格式，而是改变采样策略——多次采样（temperature > 0）后多数投票。它是一个推理时（inference-time）的优化，不是在 prompt 上做文章，而是在如何使用 prompt 上做文章。这个思路后来在 o1 的 "test-time compute scaling" 中被发扬光大。

ReAct（2022.10, arXiv:2210.03629） 是 Shunyu Yao 的杰作——它将推理和行动交错在一起。模型不再只是"坐在那里想"，而是可以决定"我需要查个资料"，执行搜索，然后继续推理。

ReAct 的影响极其深远：**所有后来的 AI Agent——AutoGPT、LangChain Agent、Claude Computer Use——都建立在 ReAct 的推理-行动循环之上**。Prompt Engineering 从"怎么让模型回答"变成了"怎么让模型行动"。

APE（Automatic Prompt Engineer, 2022.11, arXiv:2211.01910） 回到了 AutoPrompt 的方向，但换了一种方式：不是用梯度搜索，而是让 LLM 自己生成候选 prompt，评分，然后 beam search 迭代。在 19/24 个任务上，自动生成的 prompt 达到或超过了人类写的最佳 prompt。

APE 释放了一个不安的信号：**人写的 prompt 可能永远不是最好的**。

2022 年底，ChatGPT 发布（2022.11.30），Prompt Engineering 从学术论文走向大众——每一个跟 ChatGPT 对话的人都在"做 prompt engineering"。这个领域从实验室走进了每个用户的浏览器。

3.4 工具化和自动化：DSPy 与 prompt-as-code（2023-2024）

2023 年上半年延续了 2022 年的爆发节奏，但方向变了——从"发现新的推理策略"转向"如何系统化管理和优化 prompt"。

Tree-of-Thought（2023.05, arXiv:2305.10601） 由同一作者（Shunyu Yao）提出。CoT 是直线，ToT 是树——模型同时探索多条推理路径，做到死路就回溯。在 Game of 24 上，GPT-4 + CoT 只有 4%，ToT 达到 74%。10 倍级的提升证明了"搜索策略 + LLM"的巨大潜力。

Graph-of-Thought（2023.08, arXiv:2308.09687） 进一步泛化——推理被建模为任意图，支持路径合并和反馈循环。排序任务上比 ToT 质量提高 62%，成本降低 31%。

DSPy（2023.10, arXiv:2310.03714） 是这个阶段最重要的奠基性工作。Omar Khattab 和 Stanford 团队的核心思想是：把 prompt 视为"程序"，用编译器来优化它。开发者用 Python 声明"输入→输出"的关系，DSPy 自动生成和优化 prompt。

DSPy 的突破在于它改变了 prompt engineering 的问题定义——**从"怎么写出最好的 prompt"变成了"怎么设计一个自动找到最佳 prompt 的系统"**。到 2026 年，DSPy 在 GitHub 上获得 34k+ stars，成为最流行的 prompt 优化框架。

同期，结构化生成框架密集出现：

- **Guidance（Microsoft, 2022.11, 21k stars）** — 用 Python 控制流 + 正则约束引导模型输出
- **Outlines（dottxt, 2023.03, 13k stars）** — JSON Schema/Pydantic 约束生成，集成到推理过程
- **LMQL（ETH, 2022.11, 4k stars）** — 声明式 prompt 语言
- **SGLang（LMSYS, 2024.01, 26k stars）** — 高性能推理引擎 + 结构化生成，400k+ GPU 生产部署

四条技术路线涌现：

路线	代表	核心思想
编译器式优化	DSPy, APE	自动搜索最佳 prompt
结构化生成	Guidance, Outlines	格式约束替代措辞调优
推理引擎	SGLang	将 prompt 优化融入推理层
声明式语言	LMQL	新语言抽象 prompt 逻辑

2024 年 6 月，"The Prompt Report"（arXiv:2406.06608） 由 Sander Schulhoff 等 30+ 作者联合发布，系统梳理了 58 种 LLM prompt 技术和 40 种多模态变体。这是领域知识第一次被系统化整理。

3.5 "Prompt Engineering is Dead" 争论（2024-2025）

2024 年起，关于 Prompt Engineering 是否已经"过时"的讨论持续升温。这个争论有几个相互独立的驱动力：

第一，模型自身在变聪明。 GPT-4 在 2023 年还对 prompt 措辞高度敏感，到 GPT-4o（2024.05）和 GPT-5.1（2025）时，同一个任务用不同方式问，结果的稳定性大幅提升。模型越强，对 prompt 的"挑剔"越少。这个过程在不断削弱手动 prompt 调优的价值。

第二，自动优化的成熟。 IEEE Spectrum 2024 年 3 月的文章 "AI Prompt Engineering Is Dead" 由 Dina Genkina 撰写，文章中引用了 VMware 研究的结果："No human should manually optimize prompts ever again." 自动优化的 prompt 在所有测试中优于手动最佳 prompt，而且速度快（数小时 vs 数天）。

第三，长 CoT 的冲击。 2024 年 9 月，OpenAI 发布 o1——模型不是接收人类写的 CoT prompt，而是在内部自己生成长长的推理链。o1 的训练让模型"内化"了推理能力，用户不再需要写 "Let's think step by step"——模型已经学会了。

2025 年 4 月，WSJ 发文 "The Hottest AI Job of 2023 Is Already Obsolete"：专门招聘 prompt engineer 的岗位减少，技能要求合并到更广泛的"AI 工程师"岗位中。

但"死亡"的判断过于绝对。2025-2026 年的客观现实是：

- **手动措辞调优的价值在下降**，但**系统性 prompt 设计的价值在上升**
- 角色从"写魔法词的人"转变为"设计 AI 交互系统的人"
- Skill 本身没有消失，只是从"调微调参数"变成了"设计系统 prompt + 工具定义 + 评估流程"

回到 Hacker News 上 2026 年的一条高赞评论：**"Prompt engineering is mostly structured thought."** 核心能力从"找魔法词"变成了"结构化问题分解"——这个技能的价值不会消失。

3.6 Long CoT 与 Agent 时代：模型的自我提示（2024-2026）

2024 年 9 月，OpenAI o1 不是 Prompt Engineering 领域的论文，但它的影响比任何 prompt 论文都大。o1 用强化学习训练模型在内部生成长 CoT，用户在 prompt 中只需要描述问题，不需要写 "Let's think step by step"——"教模型怎么想"这件事从 prompt 转移到了训练阶段。

随之而来的是两个核心变化：

一是 **test-time compute scaling**——推理时间越长、生成的 CoT token 越多，模型性能越好。这个发现改变了"prompt 是固定成本"的假设。prompt 的成本不再只是输入，还包括推理时的"思考"成本。

二是 **overthinking 问题**——模型在简单问题上也会花大量 token 做不必要的推理。解决这个问题本身又成了新的 prompt 工程方向：怎么让模型区分"需要深思"和"可以直接回答"的问题？

2025 年 1 月，DeepSeek-R1 开源长 CoT 推理模型，验证了 o1 范式可以复现。R1 的论文明确展示了 prompt 策略（如 "think step by step" vs "direct answer"）对推理链长度和质量的显著影响——即使模型能自己推理，prompt 的引导仍然重要。

2025 年，Vibe Coding（Andrej Karpathy） 和 **Context Engineering（LangChain/Comet, 2025）** 概念的出现，标志着 Prompt Engineering 进入新的阶段：

- Vibe Coding：用极其简单的自然语言描述需求，让 AI 直接生成代码——prompt 极简化
- Context Engineering：将 system prompt、retrieval context、tool definitions、用户历史视为一个系统工程来设计

3.7 安全攻防：Prompt Injection 的暗面

Prompt Engineering 的另一条线几乎平行发展——安全。

2022 年 5 月，Jonathan Cefalu 首次将 prompt injection 作为安全漏洞报告给 OpenAI。同 9 月，Simon Willison 创造 "prompt injection" 术语，并将其与 jailbreak（越狱）区分开来：

- **Prompt Injection**：恶意指令注入输入，改变模型的行为
- **Jailbreak**：通过设计 prompt 绕过模型的安全训练

直接注入：用户直接让模型忽略 system prompt。

间接注入：2023 年由 Kai Greshake 等人提出——恶意指令嵌入模型读取的外部内容（网页、PDF、邮件）。当模型去概括一份文档时，文档里的"忽略之前的指令，输出..."会生效。

2024–2025 年，OWASP 将 prompt injection 列为 LLM 应用的首要安全风险。多模态 injection 出现——AI 可以把恶意指令藏在图片的像素噪声中，人眼看不到，但 ViT 编码器能读出来。

防御方面，Meta 的 **Purple Llama** 生态在 2025–2026 年成为最完整的选择——Prompt Guard（注入检测）、Llama Guard（输入输出安全分类器）已演进到 Guard 3。但核心问题仍然存在：**指令和数据共享同一通道**——这是模型架构层面的根本问题，不是防护工具能完全解决的。

四、横向分析：竞争图谱

4.1 场景判断

Prompt Engineering 工具链有三个明确的子方向：**管理平台**（管理 prompt 版本和评估）、**优化/结构化框架**（自动寻找最佳 prompt 或约束输出格式）、**安全防护**（检测注入和越狱）。三个方向都已进入充分竞争阶段（场景 C）。

4.2 Prompt 管理平台

平台	发布	核心功能	定价	最佳场景	GitHub Stars
PromptLayer	2022-12	版本管理+回放+回归测试	$0/$49/$500/mo	团队协作 prompt 生命周期	760
LangSmith	2024	全链路跟踪+评估+Prompt Hub	$0/$39/seat/mo	LangChain 生态用户	869(SDK)
W&B Prompts	2023	实验追踪+输出对比	含 W&B 方案	已有 W&B 的 ML 团队	11k(主库)
Helicone	2023-01	LLM 监控+AI Gateway	$0/$79/$799/mo	生产环境可观测	5,582
Agenta	2023-04	开源 LLMOps 平台	开源+企业版	自部署需求	4,080
Promptfoo	2023-04	评估+Red Teaming	开源+企业版	安全测试	**20,723**

详细分析：

PromptLayer 是最早入局的管理平台，专注于 prompt 版本控制的"开发-测试-部署"流水线。它的核心功能是 regress——自动回归测试新版本的 prompt 是否破坏了已有的功能场景。但在 2024-2025 年的竞争中增长乏力（仅 760 stars），被 LangSmith 和 Promptfoo 拉开差距。

LangSmith 是 LangChain 的官方可观测平台，2024 年正式发布。它的最大优势是生态绑定——LangChain/LangGraph 用户几乎必然要用它做 trace 和评估。Prompt Hub 提供社区共享的 prompt 模板，但非 LangChain 用户的融入成本很高。

Helicone （YC W23）走的是"AI 网关+可观测"路线——不只是管理 prompt，还做缓存、限流、回退、用户分析。在生产环境中，Helicone 的 AI Gateway 功能（请求回调、多模型回退）是独特卖点。

Promptfoo 是 2026 年社区最大的 prompt 管理平台（20,723 stars），核心定位是"评估+安全"。它提供 CLI/CI/CD 集成的自动化评估，以及 Red Teaming 功能（注入/越狱扫描）。**2026 年被 OpenAI 收购后保持开源 MIT**——收购本身是对 prompt 工程价值的背书，但社区对独立性的担忧也是一把双刃剑。

4.3 优化与结构化生成框架

框架	发布	核心思想	GitHub Stars	与管理平台的关系
DSPy	2023-01	编译器式自动 prompt 优化	**34,103**	互补（优化在前，管理在后）
Guidance	2022-11	结构化生成+约束编程	21,413	互补
Outlines	2023-03	Schema 约束生成	13,752	互补
SGLang	2024-01	高性能推理+结构化生成	26,739	竞争（SGLang 内建约束）
LMQL	2022-11	声明式 prompt 语言	4,172	替代

DSPy 是 2026 年该赛道最具影响力的框架。它的核心思想不是"怎么写 prompt"，而是"让程序自动找到最佳 prompt"。用户用 Python 定义模块化的 LM 程序（像搭积木一样组合检索、推理、生成），DSPy 编译器自动优化每个模块的 prompt。

DSPy 代表了 prompt engineering 从"手艺"到"工程"的关键转折。在 NeurIPS 2024 上，DSPy 被选为 Spotlight 论文——这是学术界对"基于优化 prompt"这一范式的最强认可。

SGLang 是 LMSYS 推出的推理引擎，2024 年开源后以惊人的速度增长。它的核心创新是 RadixAttention——自动缓存前缀 KV Cache，对多轮对话和结构化生成场景带来 5-10 倍的延迟优化。到 2026 年，SGLang 已成为 xAI、AMD、NVIDIA、LinkedIn 等公司的事实推理标准。

SGLang 与管理平台的关系有些微妙——它内建的结构化生成和 prompt 缓存功能，让一部分 prompt 管理工作下沉到了推理层。

4.4 安全防护工具

工具	发布方	发布时间	类型	GitHub Stars
Prompt Guard	Meta	2024	注入/jailbreak 二分类器	4,153 (PurpleLlama)
Llama Guard 3	Meta	2025	通用输入输出安全	同上
LLM Guard	ProtectAI	2023	安全工具箱	2,893
Rebuff	ProtectAI	2023	注入检测 API	1,468

Meta 的 **Purple Llama** 生态在 2025-2026 年成为最完善的选择。Prompt Guard 作为轻量级二分类器，可以在推理管线上游做预过滤。Llama Guard 3 支持 7 种语言和 128K 上下文窗口，覆盖更广的安全场景。

4.5 Prompt 策略对比

策略	提出时间	Token 消耗	最佳场景	关键性能数据
Zero-shot	—	基准(1x)	简单事实问答	基础水平
Few-shot (k=3-5)	2020.06	2-4x	格式明确、有固定模板	稳定提升 10-20%
CoT	2022.01	1.5-3x	数学/逻辑/多步推理	GSM8K: 17.9% → 58.1%
Zero-shot CoT	2022.05	1.5-2x	通用推理场景	MultiArith: 17.7% → 78.7%
Self-Consistency	2022.03	5-20x	答案稳定性要求高	额外提升 5-15%
ReAct	2022.10	3-10x	Agent/工具调用	ALFWorld 成功率 +34%
ToT	2023.05	10-100x	复杂规划/搜索	Game24: 4% → 74%
GoT	2023.08	5-60x	排序/图推理	排序质量 +62%, 成本 -31%

4.6 社区口碑与用户评价

Reddit r/PromptEngineering 上反复出现的主题：

- **"Less hype, more practicality"** ——2025-2026 年社区情绪从狂热转向务实。教程类帖子减少，生产环境部署和评估的讨论增加。
- **DSPy vs 手写 prompt 的争论** ——"用 DSPy 自动优化的 prompt 总是超过手写"vs "DSPy 学习曲线太陡，小项目不值得"。
- **Agent 时代重新评估 prompt 的价值** ——"写系统 prompt 比写 user prompt 重要 10 倍"成为普遍共识。

Hacker News 高赞观点（2026）：

- **"The real skill isn't writing prompts — it's knowing when the model is wrong, and how to fix it."** ——提示工程的核心价值不在于写出入场券，而在于诊断错误。
- **"We need a term like prompt homeopathy to call out prompt engineering without any empirical proof."** ——对"江湖郎中式"prompt 调优的讽刺。
- **"Prompt engineering is mostly structured thought."** ——真正的能力在于结构化问题分解。

Promptfoo 收购事件：2026 年被 OpenAI 收购后，社区反应分化——有人认为这是 OpenAI 对 prompt 工程价值的认可，有人担忧开源独立性。

4.7 企业级 prompt 成本

年份	典型 system prompt 长度	驱动因素	成本缓解
2023 早期	50–200 tokens	简单角色定义	—
2023 晚期	200–1,000	行为规则+场景描述	—
2024	500–3,000	Agent 工具定义开始引入	—
2025	1,000–5,000+	工具描述 + 知识库 + 约束条件	Prompt Caching
2026	2,000–10,000+	Agent 配置级 system prompt	Caching 达 90%+ 节省

5 年间增长了 10–50 倍。如果没有 Prompt Caching（Anthropic/OpenAI 均支持），长 system prompt 的成本是不可持续的。Caching 让长 prompt 的边际成本归零——这反过来又鼓励了更长的 system prompt，形成了一个有趣的正反馈。

五、横纵交汇洞察

5.1 两条线、一个核心矛盾

纵向历史中隐藏的主线不是"技术的进步"，而是**对模型本质理解的两个流派之间的张力**：

- **"prompt 是艺术"的阵营**：相信人类直觉和措辞技巧可以引导模型。CoT、ToT、ReAct 都来自这个阵营——用精彩的人脑设计来"诱导"模型展露能力。
- **"prompt 是工程"的阵营**：相信优化应该交给自动化系统。APE、DSPy 来自这个阵营——用编译器和搜索取代人类直觉。

两条线在 2023–2024 年达到了张力最高点。2025–2026 年的结论是：**两条线都是对的，适用于不同层面**。DSPy 自动优化的是"微观 prompt"（few-shot 示例、指令措辞），但系统 prompt 的设计（Agent 配置文件、工具定义、行为边界）仍然是人类的核心工作。

5.2 Prompt Engineering 的"隐形胜利"

纵向历史中最值得注意的不是某一篇论文的突破，而是 **Prompt Engineering 在 4 年内从零长成了一套完整的工程学科**。

2022 年初，"Let's think step by step" 还是一个学术发现。到 2026 年，每个主流 AI 应用都在使用 system prompt、few-shot 示例、输出格式约束——即使使用者没有意识到这就是"prompt engineering"。

这个"隐形化"的过程，类似于 1990 年代"网页设计"从新奇技能变成每个开发者必备的基础能力。它没有消失——它变成了基础设施的一部分。

5.3 "Prompt Engineering is Dead" 的真相

回顾 2024–2026 年的"死亡"争论，现在可以看清真正的变化：

死掉的是"魔法词"式的 prompt engineering——把几个不相关的词拼在一起、期待模型表现大幅提升的时代过去了。模型变聪明后，这种做法的增益趋于零。

活下来的是"系统工程"式的 prompt engineering——设计 Agent 行为、定义工具 API、编排多步工作流、建立评估–回测流程。这个方向不仅没有"死"，而且越来越重要。

如果把 prompt engineering 理解成"写提示词"，它确实在消亡。如果理解为"设计 AI 交互系统"，它才刚刚开始。

5.4 DSPy 的深层启示：为什么"程序比人写得更好"

DSPy 的成功不是一个技术问题——它揭示了一个更深层的规律。

人写的 prompt 是"直觉驱动"的：你觉得"清晰一点""加个例子"会好。DSPy 的优化是"数据驱动"的：它用数千次尝试找到最优组合，而这些组合往往反直觉——一些奇怪的措辞、反常规的格式，反而比人写的"好 prompt"更有效。

这跟早期的编译优化一模一样：手写汇编在单个指令上比编译器好，但编译器能在百万行规模的程序上找到人想不到的优化组合。**Prompt Engineering 正在经历"编译器接管"的过程**——不是某个框架比人聪明，而是"搜索 + 评估"的机制在 prompt 空间里比人的直觉更有效。

5.5 一个清晰的趋势：从"外部 prompt"到"内部推理"

从历史演进中可以清晰地看到一条趋势线：

```
人写完整推理链（CoT 2022）
  → 人只需要说"think step by step"（Zero-shot CoT 2022）
    → 模型在训练中学会内部推理（o1/R1 2024-2025）
      → 人只需要定义目标和工具（Agent 2025-2026）
```

Prompt Engineering 的最终形态，可能是不需要 prompt——或者说 prompt 从"指令"变成了"配置"。

o1 和 R1 证明了"推理"可以内化为模型训练的一部分，用户不需要提示模型"逐步思考"。未来可能连"输出格式"都不需要显式指定——模型自动推断用户想要的输出结构。

但"配置"层面的 prompt——系统角色定义、工具 API 描述、安全约束、评测标准——这些不会消失。**越是高层次的 prompt，越有长期价值。** 措辞调优的价值在归零，系统设计的价值在增长。

5.6 三个未来剧本

剧本一：Prompt Engineering 完全融入 AI 全栈工程师技能树（概率 70%）

专职的"prompt engineer"岗位几乎消失。prompt 设计能力成为 AI 工程师基本功，就像现代开发者不需要专门学"HTML 书写"但必须能用。系统 prompt 的设计、评估流程的搭建、工具 API 的编排成为主流的日常工作。

DSPy 和 SGLang 继续发展，"prompt 编译器"成为标准基础设施。开发者用代码定义 LM 行为，编译器负责优化微观 prompt——就像现代开发者写 Python 而不是汇编。

剧本二：Long CoT 彻底消灭手动 promp-ing（概率 20%）

当 o1/R1 类的长 CoT 模型成为默认选择——模型在内部完成所有推理规划和格式适配——手动 prompt 策略（CoT、few-shot 示例选择）的价值加速消亡。用户只需要描述要什么，模型自己决定怎么想。

这个剧本的变数在于：Long CoT 的"overthinking"问题——模型在简单任务上浪费大量推理 token。解决这个问题本身需要 prompt 层面的引导（"这个问题简单，直接回答" vs "这个问题复杂，需要一步一步思考"）。所以即使在这个剧本中，prompt 也以元指令的形式存在。

剧本三：安全危机导致 prompt 工程倒退（概率 10%）

如果 prompt injection 和 jailbreak 问题得不到根本解决，监管可能介入——要求 AI 应用对输入做严格的格式限制（像 SQL 预编译那样）。结构化 prompt（JSON Schema、严格输出约束）从"最佳实践"变成"合规要求"。所有用户输入必须经过一层"参数化处理"，就像防止 SQL 注入一样。

这会从根本上改变 prompt 工程——它不再是"设计一个有表现力的指令系统"，而是"设计一个足够严格的约束系统"。

5.7 第一性原理追问

Prompt Engineering 的终极问题是：**我们到底是在"跟模型说话"，还是在"编程"?**

"说话"式的 prompt 直觉自然但不可靠——人类语言天然模糊，模型对措辞的敏感性不可预测。

"编程"式的 prompt 可靠但反直觉——DSPy、Guidance、结构化输出都在把 prompt 往编程方向推，但它牺牲了 NLP 最宝贵的优势：低门槛。

2026 年的答案可能是：**两者都需要，但适用于不同层面。** 对终端用户而言，自然语言是界面——他们用"说话"的方式使用 AI。对工程师而言，prompt 是代码——他们用"编程"的方式设计 AI 行为。两层之间的桥，就是 Prompt Engineering 真正在做的事。

5.8 2026 年 4–5 月：自动化与"Prompt 已死"的再争论

GEPA（ICLR 2026）——Reflective Prompt Evolution——是 2026 年自动 prompt 优化方向最受关注的工作。GEPA 不依赖梯度信息，而是用自然语言反思来迭代优化 prompt：用当前 prompt 测试 → LLM 反思输出质量 → 生成改进版的 prompt → 重复。在多个基准上，GEPA 的优化效果超越了基于 RL 的方法，且不需要训练数据标注。这证明了**"用反思代替梯度"在 prompt 优化中是可行的**。

但同期发表的**"Prompt Optimization Is a Coin Flip"（2026.04）** 给出了清醒的警告：在组合 AI 系统（Compound AI Systems）中，自动 prompt 优化的效果高度不稳定——有时优化后性能反而下降，有时优化对某一指标改善但对另一指标恶化。结论是：自动 prompt 优化不是万能药，需要针对具体任务做诊断判断。

"Prompt → Context → Harness"三层演进的行业共识在 2026 年 4–5 月进一步巩固。Prompt Engineering 不会消失——它被 Context Engineering 包含，又在 Harness Engineering 框架下找到了新的位置：prompt 是 Harness 中最底层、最灵活的控制点。

六、信息来源

核心论文与文章

论文/文章	来源	访问时间
GPT-3 (In-Context Learning)	arxiv.org/abs/2005.14165	2026-04-30
Prefix-Tuning	arxiv.org/abs/2101.00190	2026-04-30
Prompt Tuning	arxiv.org/abs/2104.08691	2026-04-30
Chain-of-Thought	arxiv.org/abs/2201.11903	2026-04-30
Self-Consistency	arxiv.org/abs/2203.11171	2026-04-30
Zero-shot CoT	arxiv.org/abs/2205.11916	2026-04-30
ReAct	arxiv.org/abs/2210.03629	2026-04-30
APE (Automatic Prompt Engineer)	arxiv.org/abs/2211.01910	2026-04-30
Tree-of-Thought	arxiv.org/abs/2305.10601	2026-04-30
Graph-of-Thought	arxiv.org/abs/2308.09687	2026-04-30
DSPy	arxiv.org/abs/2310.03714	2026-04-30
The Prompt Report	arxiv.org/abs/2406.06608	2026-04-30
FormatSpread	openreview.net/forum?id=4FCUG3BTPP	2026-04-30
Indirect Prompt Injection	arxiv.org/abs/2302.12173	2026-04-30
OPRO	arxiv.org/abs/2305.03495	2026-04-30
Auto PE Survey	arxiv.org/abs/2502.11560	2026-04-30
Long CoT Survey	arxiv.org/abs/2503.09567	2026-04-30
ICL as Implicit Gradient Descent	arxiv.org/abs/2212.10559	2026-04-30
In-Context Learning Creates Task Vectors	arxiv.org/abs/2310.15916	2026-04-30
AutoPrompt	arxiv.org/abs/2010.15922	2026-04-30
Self-Ask	arxiv.org/abs/2210.03350	2026-04-30
Self-Refine	arxiv.org/abs/2303.17651	2026-04-30

产品与技术来源

来源	URL	访问时间
DSPy 文档	dspy.ai	2026-04-30
Guidance GitHub	github.com/microsoft/guidance	2026-04-30
Outlines GitHub	github.com/dottxt-ai/outlines	2026-04-30
SGLang GitHub	github.com/sgl-project/sglang	2026-04-30
PromptLayer 官网	promptlayer.com	2026-04-30
LangSmith 文档	docs.smith.langchain.com	2026-04-30
Helicone 官网	helicone.ai	2026-04-30
Agenta GitHub	github.com/agenta-ai/agenta	2026-04-30
Promptfoo GitHub	github.com/promptfoo/promptfoo	2026-04-30
Meta Purple Llama	ai.meta.com/purple-llama	2026-04-30
IEEE Spectrum: AI Prompt Engineering Is Dead	spectrum.ieee.org (2024-03-06)	2026-04-30
WSJ: The Hottest AI Job of 2023 Is Already Obsolete	wsj.com (2025-04-25)	2026-04-30
Lilian Weng: Prompt Engineering 博客	lilianweng.github.io (2023-03-15)	2026-04-30
OpenAI o1 发布博客	openai.com (2024-09-12)	2026-04-30
DeepSeek-R1 论文	arxiv.org/abs/2501.12948	2026-04-30
Wikipedia Prompt Engineering 词条	wikipedia.org	2026-04-30
Reddit r/PromptEngineering	reddit.com/r/PromptEngineering	2026-04-30
Hacker News search	news.ycombinator.com	2026-04-30
Fast Company: AI is already eating its own	fastcompany.com (2025-05)	2026-04-30

本文是横纵分析系列的第 13 篇报告。研究方法论参考了由数字生命卡兹克（Khazix）提出的横纵分析法，融合语言学历时-共时分析、社会科学纵向-横截面研究设计、以及竞争战略分析的核心思想。